71. Генерация случайных чисел

Всякие генераторы случайные числа очень полезны в некоторых видах программ, в частности, в игре, программах научного или статистического моделирования. Возьмем, к примеру, игру без **рандомных** или **случайных событий** — монстры всегда будут атаковать вас одинаково, вы всегда будете находить одни и те же предметы/артефакты, монеты тяжелы и выдают золото/опыт по будут меньше и т.д. В общем, скоро такой игры не очень интересен и вряд ли вы будете в нее долго играть.

Оглавление:

1. Генератор псевдослучайных чисел
2. Функция srand() и rand()
3. Стартовое число и последовательности в ГПСЧ
4. Генерация случайных чисел в заданном диапазоне
5. Какой ГПСЧ является хорошим?
6. Почему rand() является посредственным ГПСЧ?
7. Отладка программ, использующих случайные числа
8. Рандомные числа в C++11

Генератор псевдослучайных чисел

Так как не генерировать случайные числа? В реальной жизни мы часто бросаем монетку (орел/решка), кости или перетасовываем карты. Эти событие включают в себя так много физических переменных (например, силу тяжести, трение, сопротивление воздуха и д.), что они становятся непредсказуемыми (непредсказуемыми/которые и выдают результаты, которые вы всех смысла является случайным.

Однако компьютеры не предназначены для использования физических переменных — они не могут подбросить монетку, бросить кости или перетасовать реальные карты. Компьютеры живут в контролируемом электрическом мире, где есть только два значения (true и false), которые по существу между ними нет. Это такой прекрасный контролируемый для получения прогнозируемых результатов. Когда мы говорим компьютеру посчитать, сколько будет 2 x 2 , мы всегда хотим, чтобы ответом было 4 (не 3 и не 5).

Следовательно, компьютеры неспособны генерировать случайные числа. Вместо этого они могут имитировать случайность, что достигается с помощью генераторов псевдослучайных чисел.

Генератор псевдослучайных чисел (ГПСЧ) — это программа, которая принимает стартовое/начальное значение и выполняет с ним определенные математические операции, чтобы конвертировать его в другое число, которое своем не похоже со стартовым. Затем программа использует новое сгенерированное значение и выполняет с ним те же математические операции, что и с начальным числом, чтобы конвертировать его в еще одно новое число — третье, которое не связано ни с первым, ни со вторым. Затем этот алгоритм и повторяет сгенерированное значение, программа может генерировать целый ряд новых чисел, которые будут казаться случайными (при условии, что алгоритм будет достаточно сложным).

На самом деле, написать простой ГПСЧ не так уж и сложно. Вот небольшая программа, которая генерирует 100 рандомных чисел:

```
#include <iostream
int PRNG(
{
    // Static variables keep their values between function calls
    static unsigned int seed = 4561;
    // Modulus, multiplier, and increment from Numerical Recipes
    const unsigned int m = 2147483648, a = 1664525, c = 1013904223;
    seed = (a*seed + c) % m;
    // Return current value
    return seed;
}

int main()
{
    // Print out 100 random numbers
    for (int count = 0; count < 100; ++count)
    {
        std::cout << PRNG() << "\t";

        // Print newline every 5 numbers
        if (count % 5 == 4)
            std::cout << "\n";
    }
}
```

Результат выполнения программы:

```
18256   4675    3486    6217    27484
975     28866   15325   25968   2987
12974   26465   13684   18471   19898
12265   23424   23667   16878   3785
22412   9727    1490    773     18648
1419    8926    3473    28988   31515
5618    13485   24908   1699    24318
22769   9168    18287   32258   12597
19912   24387   29454   5857    19924
11551   13898   3145    9184    4387
24358   6873    28468   2655    23866
16229   28904   6635    9822    31215
18756   16247   17994   19869   22544
31491   16218   13953   23588   19599
2682    13669   13864   13339   13166
14637   26104   12711   11898   26797
27712   17755   32646   28461   18588
28351   9874    31685   31328   11851
19128   26193   612     983     30378
26333   24688   28515   8118    32585
```

Каждое число является случайным по отношению к предыдущему. Главный недостаток этого алгоритма — его примитивность.

Функции srand() и rand()

Языки C и C++ имеют свои собственные встроенные генераторы случайных чисел. Они реализованы в виде отдельных функции, которые находятся в заголовочном файле cstdlib.

- **Функция srand()** устанавливает передаваемое пользователем значение в качестве стартового. srand() следует вызывать только один раз — в начале программы (обычно в первой части функции main()).
- **Функция rand()** генерирует следующее случайное число на основе предыдущего/последнего. Оно будет находиться в диапазоне от 0 до RAND_MAX (константа в cstdlib, значением которой является 32767).

Вот пример программы, в которой используется обе эти функции:

```
#include <cstdlib
#include <iostream
int main()
{
    srand(4561); // seed random number generator
    // Print out 100 random numbers
    for (int count = 0; count < 100; ++count)
    {
        std::cout << rand() << "\t";

        // Print newline every 5 numbers
        if (count % 5 == 4)
            std::cout << "\n";
    }
}
```

Результат выполнения программы:

```
14867   24688   8872    25432   21865
17185   18997   28178   16397   30572
22339   31588   1553    124     779
6687    23563   5754    23895   16527
19888   18792   17377   28696   8131
18671   27855   8870    4808    31268
31854   5873    19422   23885   18222
3631    15884   18857   30853   32018
31867   24585   16240   14389   31829
13403   13442   5385    9644    9341
11478   189     5570    29711   25679
1366    24567   25223   118     24352
24135   459     7236    17918   3338
24841   29908   24838   1894    13193
18534   8192    6868    4791    1351
14521   31493   4533    11189   7971
5118    19884   1747    23543   389
28713   24884   16478   22142   27238
6261    12836   5618    17862   31042
18638   7427    23807   25546   22229
```

Стартовое число и последовательности в ГПСЧ

Если вы запустите вышеприведенную программу (генерация случайных чисел) несколько раз, то заметите, что в результате всегда выходится одни и те же числа! Это означает, что, хотя каждое число в последовательности кажется случайным относительно предыдущего, все последовательность не является случайной вообще! А это, в свою очередь, означает, что наша программа полностью предсказуема (одни и те же значения вывода приводят к одним и тем же значениям вывода). Выбрает случае, когда это может быть полезно или даже желательно (например, если вы хотите, чтобы научная симуляция повторялась, или вы пытаетесь исправить причину сбоя вашего генератора случайных подключений в игре).

Но в большинстве случаев это не совсем то, что нам нужно. Если вы пишете игру типа Ml-Ed (игру у пользователя есть 10 попыток угадать число, а компьютер говорит вам, насколько это предположение близки или далеки от реального числа), вы бы не хотели, чтобы программа выбирала одни и те же числа каждый раз. Поэтому давайте более подробно рассмотрим, почему это происходит и как это можно исправить.

Помните, что каждое новое число в последовательности ГПСЧ генерируется исходя из предыдущего определенных способов. Таким образом, при постоянном начальном числе ГПСЧ всегда будет генерировать одну и ту же последовательность! В программе, приведенной выше, последовательность чисел всегда начинается, так как стартовое число всегда равно 4561.

Чтобы это исправить нам нужен способ выбрать стартовое число, которое не будет фиксированным значением. Первое, что приходит на ум — использовать рандомное число! Это хорошая мысль, но если нам нужно случайное число для генерации случайных чисел, то это какой-то замкнутый круг, или не кажется? Оказывается, нам не обязательно использовать случайное стартовое число — нам просто нужно выбрать что-то, что будет меняться каждый раз при новом запуске программы. Затем мы сможем использовать часть ГПСЧ для генерации уникальной последовательности рандомных чисел исходя из уникального стартового числа.

Общепринятым решением является использование системных часов. Каждый раз, при запуске программы, время будет другим. Если мы будем использовать значение времени в качестве стартового числа, то наша программа всегда будет генерировать разные последовательности чисел при каждом новом запуске!

В языке C есть **функция time()**, которая возвращает в качестве времени общее количество секунд, прошедшее от полночи 1 января 1970 года. Чтобы использовать эту функцию, нам просто нужно подключить заголовочный файл ctime, а затем инициализировать функцию srand() вызовом функции time(0) :

Вот вышеприведенная программа, но уже с использованием функции time() в качестве стартового числа:

```
#include <cstdlib
#include <iostream
#include <ctime // for function time()
int main()
{
    srand(static_cast<unsigned int>(time(0)) // convert time to unsigned integer as seed
    for (int count = 0; count < 100; ++count)
    {
        std::cout << rand() << "\t";

        // Print newline every 5 numbers
        if (count % 5 == 4)
            std::cout << "\n";
    }
}
```

Теперь наша программа будет генерировать разные последовательности случайных чисел! Попробуйте сами.

Генерация случайных чисел в заданном диапазоне

В большинстве случаев нам не нужны рандомные числа между 0 и RAND_MAX — нам нужны числа между двумя другими значениями: a(a и вам. Например, если нам нужно симулировать бросок кубика, то диапазон значений будет равен: от 1 до 6 .

Вот небольшая функция, которая конвертирует результат функции rand() в нужный нам диапазон значений:

```
// Returns a random number between min and max
int getRandomNumber(int min, int max)
{
    static const double fraction = 1.0 / (static_cast<double>(RAND_MAX) + 1.0);
    // static used for efficiency, so we only calculate this value once
    return static_cast<int>(rand() * fraction * (max - min + 1) + min);
}
```

Чтобы симулировать бросок кубика, вызовем функцию: getRandomNumber(1, 6) .

Какой ГПСЧ является хорошим?

Как мы уже говорили, генератор случайных чисел, который мы написали выше, не является очень хорошим. Сейчас рассмотрим почему.

Хороший ГПСЧ должен иметь ряд свойств:

Свойство №1: ГПСЧ должен генерировать каждое новое число с примерно одинаковой вероятностью. Это называется **равномерным распределением**. Если некоторые числа генерируются чаще, чем другие, то результат программы, использующей ГПСЧ, будет предсказуем! Например, предположим, вы пытаетесь написать генератор случайных предметов для игры. Вы выбираете случайное число от 1 до 10, и если результат будет 10, игрок получает крутой предмет вместо среднего. Шансы должны быть 1 к 10, но, если ваш ГПСЧ неравномерно генерирует числа, например, десятки генерируется чаще, чем единицы, то ваши игроки будут получать более редкие предметы чаще, чем предполагалось, и сложность, и интерес к такой игре автоматически уменьшаются.

Создать ГПСЧ, который бы генерировал равномерные результаты — сложно, и это одна из главных причин, по которым ГПСЧ, который мы написали в начале этого урока, не является очень хорошим.

Свойство №2: Между, с помощью которого генерируется следующее число в последовательности, не должен быть очевидным или предсказуемым. Например, рассмотрим следующий алгоритм ГПСЧ: next = next + 1 . У него есть равномерность, но предсказуемость — он не сможет его от примитивности и предсказуемости!

Свойство №3: ГПСЧ должен иметь хорошее диапазонное распределение чисел. Это означает, что маленькие, средние и большие числа должны возвращаться случайным образом. ГПСЧ, который возвращает все маленькие числа, а затем все большие — предсказуем и приводит к предсказуемым результатам.

Свойство №4: Период алгоритма должен превышать количество чисел, которые вам необходимы. Все ГПСЧ являются цикличными, т.е. в какой-то момент последовательность генерируемых чисел начнет повторяться. Как упоминалось ранее, ГПСЧ является детерминированным, и с одним значением между вы получаете одно и то же значение выхода. Поэтому, что произойдет, когда ГПСЧ сгенерирует число, которое уже ранее было сгенерировано. С этого момента начнется дублирование последовательности чисел между первым и последующим появлением этого числа. Длина этой последовательности называется **периодом**.

Например, вот представлены первые 100 чисел, сгенерированные ГПСЧ с плохой периодичностью:

```
112     9       138     97      64
31      152     119     86      53
20      141     108     75      42
8       138     97      64      31
152     119     86      53      20
141     108     75      42      8
138     97      64      31      152
119     86      53      20      141
108     75      42      9       138
97      64      31      152     119
86      53      20      141     108
75      42      9       138     97
64      31      152     119     86
53      20      141     108     75
42      9       138     97      64
31      152     119     86      53
20      141     108     75      42
9       138     97      64      31
152     119     86      53      20
141     108     75      42      9
```

Заметили, что он сгенерировав 9 , как второе число, а затем как шестнадцатое! ГПСЧ застревает, генерируя последовательность между этими двумя 9-ми: 9-138-97-64-31-152-119-86-53-20-141-108-75-42- (повтор).

Хороший ГПСЧ должен иметь длинный период для всех стартовых чисел. Разработка алгоритма, соответствующего этому требованию, может быть чрезвычайно сложной — большинство ГПСЧ имеют длинные периоды для одних начальных чисел и короткие для других. Если пользователь выбрал начальное число, которое имеет короткий период, то и результаты будут соответствующие.

Несмотря на сложность разработки алгоритмов, отвечающих всем этим критериям, в этой области было проведено большое количество исследований, так как разные ГПСЧ активно используются в важных отраслях науки.

Почему rand() является посредственным ГПСЧ?

Алгоритм, используемый для реализации rand(), может варьироваться в разных компиляторах, и соответственно, результаты также могут быть разными. В большинстве реализаций rand() используется Линейный Конгруэнтный Метод (сокр. **«ЛКМ»**. Если вы посмотрите на первый пример в этом уроке, то заметите, что так, на самом деле, используется ЛКМ, хоть и с жестко/подробственным полном константами.

Одним из основных недостатков rand() является то, что RAND_MAX обычно устанавливается как 32767 (15-битное значение). Это означает, что если вы захотите сгенерировать числа в более широком диапазоне (например, 32-битные целые числа), то функция rand() не подойдет. Кроме того, она не подходит, если вы захотите сгенерировать случайные числа типа с плавающей запятой (например, между 0.0 и 1.0), что часто используется в различных видеоиграх. Наконец, функция rand() имеет относительно короткий период по сравнению с другими алгоритмами.

Тем не менее, этот алгоритм отлично подходит для изучения программирования и для программ, в которых высококачественный ГПСЧ не является необходимостью.

Для приложений, где требуется высококачественный ГПСЧ, рекомендуется использовать алгоритм Вихрь Мерсенна (англ. **«Mersenne Twister»**), который генерирует отличные результаты и относительно просто в использовании.

Отладка программ, использующих случайные числа

Программы, которые используют случайные числа, трудно отлаживать, так как при каждом запуске такой программы вы будете получать разные результаты. А чтобы успешно проводить отладку программ, нужно удостовериться, что наша программа выполняется одинаково при каждом ее запуске. Таким образом, мы сможем быстро увидеть расположение ошибок и исправить этот участок кода.

Поэтому, проводя отладку программы, полезно использовать в качестве стартового числа (с использованием функции srand()) одинаковое значение (например, 8), которое вызывает наибольшее количество ошибок. Это будет гарантий того, что наша программа каждый раз генерирует одни и те же результаты (что значительно обеспечит процесс отладки). После того, как вы найдете и исправите ошибку, мы сможем снова использовать системные часы для генерации рандомных результатов.

Рандомные числа в C++11

В C++11 добавлен точно нового функционала для генерации случайных чисел, включая алгоритм Вихрь Мерсенна, а также разные виды генераторов случайных чисел (например, равномерные, генератор Poisson и тр.) Доступ к ним осуществляется через подключение заголовочного файла cstdlib. Вот пример случайных чисел в C++11 с использованием Вихря Мерсенна:

```
// C++11 random generator using Mersenne Twister
#include <cstdlib // for rand() and srand()
#include <ctime // for time()
int main()
{
    std::random_device rd;
    std::mt19937 mersenne(rd); // Initialize Mersenne Twister with random stateless seed

    // Initialize a Mersenne Twister with a static seed
    // std::mt19937 mersenne(static_cast<unsigned int>(1337));

    // Print out 100 random numbers
    for (int count = 0; count < 100; ++count)
    {
        std::cout << mersenne() << "\t";

        // Print newline every 5 numbers
        if (count % 5 == 4)
            std::cout << "\n";
    }
}
```

Вихрь Мерсенна генерирует случайные 32-битные целые числа unsigned (а не 15-битные целые числа, как в случае с rand()), что позволяет использовать гораздо больший диапазон значений. Существует такое версия (std::mt19937_64) для генерации 64-битных целых чисел unsigned!

Примечание Для пользователей Visual Studio: Реализация функции rand() в Visual Studio имеет один существенный недостаток — первое генерируемое случайное число не сильно отличается от стартового. Это означает, что при использовании time() для генерации начального числа, первое число не будет сильно отличаться/изменится от стартового и при последующих запусках. Есть простое решение: выкинуть значение функции rand() сразу же в сборсте результат. Затем вы сможете использовать rand(), как обычно в ваше программе.